

Cryptography

5 – Discrete logarithms

Gabriel Chênevert

October 2, 2025

JUNIA

Today

The discrete logarithm problem

Applications: Diffie-Hellman, ElGamal

Cryptanalysis

Recall: modular exponentiation

For a given modulus n :

$$(g, \ell) \mapsto x \equiv_n g^\ell$$

RSA: hard to recover g from x even if ℓ is known

(essentially need to factor n)

"discrete ℓ^{th} root problem"

Discrete logarithm problem

Also hard to recover ℓ from x even if g is known!

Definition (discrete logarithm)

$$\text{dlog}_n(x, g) \stackrel{\text{ord}_n(g)}{:=} \ell \iff x \equiv_n g^\ell,$$

where $\text{ord}_n(g)$ is the *multiplicative order* of g in $\mathbb{Z}/n\mathbb{Z}$, i.e. the smallest positive integer for which

$$g^{\text{ord}_n(g)} \equiv_n 1.$$

By Euler's theorem, we know in general that $\text{ord}_n(g) \mid \varphi(n)$.

Example

Compute $\text{dlog}_{2039}(781, 1769)$.

We can try a brute force approach:

ℓ	1	2	3	4	5	6	7	8	9
$g^\ell \% n$	1769	1535	1506	1180	1523	668	1111	1802	781

What about $\text{dlog}_{2039}(1617, 1769)$?

General fact

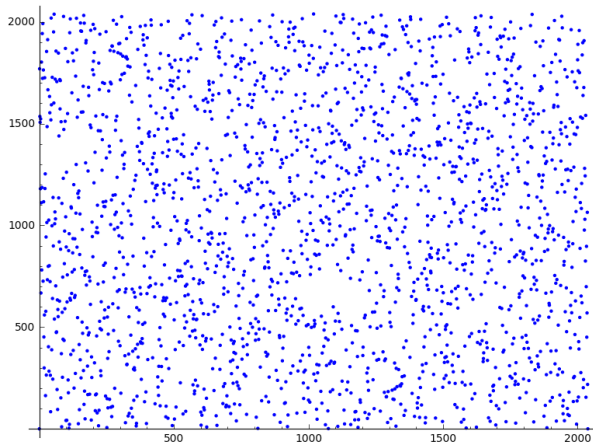
Logarithms never behave quite as well as exponentials.

(think: speed of convergence of power series, ...)

Meaning here: discrete logs can take much *longer* to compute than modular exponentials.

Information can be hidden in exponents!

Example: $x \equiv_{2039} 1769^\ell$



Today

The discrete logarithm problem

Applications: Diffie-Hellman, ElGamal

Cryptanalysis

Secret sharing

Public-key encryption provides a partial solution to the problem of setting up a shared private key for symmetric encryption on an insecure channel:

- Alice chooses secret k ,
- encrypts it with Bob's public encryption key,
- and sends it to him;
- Bob recovers k using his private decryption key.

Are there problems with that? (hint: yes, some)

”Symmetric” version

- Alice chooses k_A and sends it to Bob using his public encryption key;
- Bob chooses k_B and sends it to Alice using her public encryption key;
- Shared secret is $k := k_A \oplus k_B$.

Better: neither Alice nor Bob fully controls the final secret.

But two public encryption key pairs are needed. . .

Diffie-Hellman (1976)

- Alice and Bob agree on "safe" parameters n and g .
- Alice chooses α , computes $a \equiv g^\alpha \pmod n$ and sends it to Bob.
- Bob chooses β , computes $b \equiv g^\beta \pmod n$ and sends it to Alice.

Shared secret is

$$k \equiv g^{\alpha\beta} \pmod n \equiv a^\beta \equiv b^\alpha.$$

Diffie-Hellman problem

Eve is faced with the problem:

given a and b , recover k .

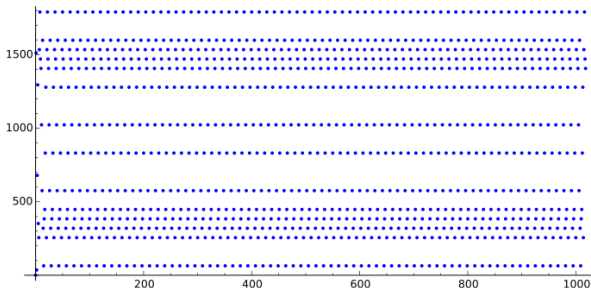
We *believe* that her best line of attack is:

- compute $\alpha = \text{dlog}_n(a, g)$ or $\beta = \text{dlog}_n(b, g)$
- then easily deduce $k \equiv_n g^{\alpha\beta}$ as Alice or Bob would.

DH caveats

- Bob should check that Alice does not provide a value of a for which the discrete log is easy (same on Alice's side)

Example: $x \equiv_{1856} 1514^\ell$



DH caveats

- Should **always** be used in conjunction with authentication to prevent *person-in-the-middle (PitM) attacks*



ElGamal cipher (1985)

Essentially Diffie-Hellman + one-time multiplicative pad

Public parameters: n and g (can be reused)

Keys:

- δ private decryption key
- $e \equiv g^\delta \pmod n$ public encryption key

Alice wants to send a message $m \in \llbracket 0, n \rrbracket$ to Bob.

Encryption

- Alice chooses random σ , computes $s \equiv_n g^\sigma$
- Computes shared secret $k \equiv_n e^\sigma$
- Computes encrypted $c \equiv_n k \cdot m$
- Sends the pair (s, c)

Decryption

Upon reception of a pair (s, c) , Bob

- Computes shared secret $k \equiv s^\delta \pmod n$
- Recovers $m \equiv k^{-1} \cdot c \pmod n$

Same caveats apply!

Today

The discrete logarithm problem

Applications: Diffie-Hellman, ElGamal

Cryptanalysis

Attacks on the DLP

or: how to compute discrete logarithms

To understand how to choose "safe" parameters n and g we need to understand how to force the DLP algorithms to be in the worst-case scenario.

Naive algorithm: brute-force the exponent

Takes at most $\mathcal{O}(\text{ord}_n(g)) \leq \mathcal{O}(n)$ steps

$\implies$ want g of large multiplicative order (hence in particular large n)

Chinese remainder theorem

If $n = n_1 \cdot n_2$ with n_1 and n_2 coprime:

$$x \equiv_n g^\ell \iff \begin{cases} x \equiv_{n_1} g^\ell \\ x \equiv_{n_2} g^\ell \end{cases}$$

If ℓ is recovered modulo $\text{ord}_{n_1}(g)$ and $\text{ord}_{n_2}(g)$, it is then easily recovered modulo

$$\text{ord}_n(g) = \text{LCM}(\text{ord}_{n_1}(g), \text{ord}_{n_2}(g)).$$

$\implies n$ should be as prime as possible

Here, this means: n should be prime!

CRT (again)

Hence take n a prime, so that $\varphi(n) = n - 1$.

Remember we are looking for a value $\ell \bmod \text{ord}_n(g) \mid \varphi(n)$.

If $\varphi(n) = n - 1$ factors, we can speed up the process by working modulo the factors.

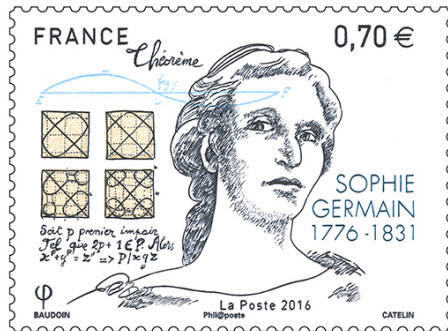
$\implies n - 1$ should be as prime as possible

Here, the best we can do is: $n = 2q + 1$ with q prime

(n : **safe** prime, q : associated **Sophie Germain** prime)

NB: We believe there are infinitely many **safe primes**.

Sophie Germain



Primitive roots

Fact: For n prime, there exists in $(\mathbb{Z}/n\mathbb{Z})^\times$ an element of order $n - 1$.

Hence, for a safe prime $n = 2q + 1$:

$$(\mathbb{Z}/n\mathbb{Z})^\times \simeq \mathbb{Z}/(n-1)\mathbb{Z} \stackrel{\text{CRT}}{\simeq} \mathbb{Z}/2\mathbb{Z} \times \mathbb{Z}/q\mathbb{Z}$$

Most nonzero elements g have multiplicative order q or $2q$.

Only two of them generate small subgroups:

$$1 \simeq (0, 0) \quad \text{and} \quad -1 \simeq (1, 0).$$

Example

$n = 2039$ is a safe prime associated to the Sophie Germain prime $q = 1019$.

What is the order of $g = 1769$?

By Lagrange's theorem, the possibilities are:

1, 2, 1019, and 2038.

By testing each modular power, we find: $\text{ord}_n(g) = 2038$.

Baby-step giant-step

Time/memory trade-off on the naive algorithm to compute $\ell \equiv_{\text{ord}(g)} \text{dlog}_n(x, g)$.

Pick some base β and write $\ell = i\beta + j$.

Baby step:

Compute and store all powers $g^j \bmod n$ for $j \in \llbracket 0, \beta \rrbracket$ in a table

Giant step:

For every $i \in \llbracket 0, \frac{\text{ord}(g)}{\beta} \rrbracket$, check if $x \cdot (g^{-\beta})^i \bmod n$ is in the above table

Baby-step giant-step

In other words: small logarithms ($< \beta$) are looked up in a pre-computed table.

In the general case for x , we look for a modified version $x \cdot (g^{-\beta})^i$ for which the logarithm is small.

$$\text{dlog}_n(x \cdot (g^{-\beta})^i, g) = j \iff \text{dlog}_n(x, g) - i\beta = j \iff \text{dlog}_n(x, g) = i\beta + j.$$

Time complexity: $\mathcal{O}(\beta) + \mathcal{O}(\frac{\text{ord}(g)}{\beta})$

Space complexity: $\mathcal{O}(\beta)$

Often take $\beta \approx \sqrt{\text{ord}(g)}$ to get time and space complexities $\mathcal{O}(\sqrt{\text{ord}(g)})$.

Example: computing $\text{dlog}_{2039}(1617, 1769)$

$$n = 2039$$

$$g = 1769$$

$$\text{ord}_n(g) = 2038$$

$$\sqrt{\text{ord}(g)} \approx 45.1$$

so we know we can take $\beta \approx 45$ and be done in at most roughly 45 giant steps

Answer: $\ell = 1397$ (easy to check!)

Other algorithms

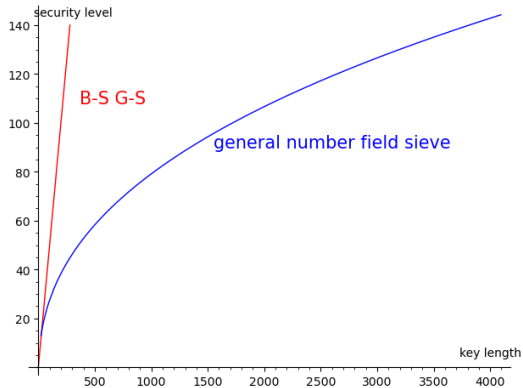
There also exists a **general-purpose probabilistic algorithm** for the DLP that takes (on average) $\mathcal{O}(\sqrt{\text{ord}(g)})$ steps (and $\mathcal{O}(1)$ memory)

Also: the General Number Field Sieve solves the *modular* DLP

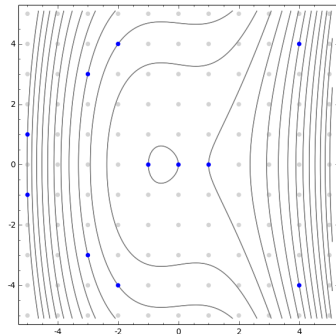
$\implies$ use same key lengths as for RSA

Current world records

Recall



Elliptic curves



Best known DLP algorithms are the *generic* ones

$\implies$ ℓ -bit security achieved by 2ℓ -bit keys ☺ (more on that tomorrow)